Bader's Event Management System

Project Documentation

King Fahd University of Petroleum and Minerals

ICS 108 — Object-Oriented Programming in Java

Java Console Application

1. Project Overview

Bader's Event Management System is a Java console application that lets a user plan, validate, and inspect events held across the KFUPM campus. Each event is tied to four pieces of campus context: an event type (Academic, Religious, Sport, or Social), a hosting department (e.g. ICS, COE, ME), a venue (Conference Hall, Lecture Hall, Sport Area, or Public Area), and a precise start/end date and time.

The program supports two input modes:

- **Manual mode** — the user is shown a menu and types commands one at a time.
- **File mode** — if a script file (e.g. Test.txt) is found at startup, the program reads input from the file silently, with prompts suppressed. This makes the program easy to grade and reproduce.

Three command codes drive the menu:

- **C** — create a new event (guided multi-step wizard).
- **SE** — show all events as a sorted summary table.
- **SI** — show one event in detail by its index.
- **Q** — quit the program.

Events are kept in chronological order automatically, and the system refuses to create an event that clashes with an existing one in the same venue.

1.1 Goals of the System

- Demonstrate object-oriented design: encapsulation, inheritance, abstraction, polymorphism, and interfaces.
- Use file I/O to load reference data (departments, venues) so the catalog is not hard-coded.
- Use exception handling to keep the program alive on bad input.
- Sort and search events using the Comparable interface.
- Keep the user interface clean and resilient — bad commands print a hint instead of crashing.

2. Architecture & Package Structure

The project is split into four cooperating packages. Each package owns a single concern, and the Main class in MAINS is the only orchestrator that talks to all of them.

Package	Responsibility	Key Classes
MAINS	Program entry point, user input handling, event types, custom exception.	Main, Input, Type, ValidationException
DEPARTMENTS	Loading and looking up the catalog of academic departments.	Department, DepartmentsManager
EVENTS	Date/time modeling, event objects, scheduling, and clash detection.	DateAndTime, Event, EventManager
VENUES	Venue hierarchy (4 subclasses) plus catalog management.	Venue (abstract), ConferenceHall, LectureHall, SportArea, PublicArea, VenueManager

2.1 Data Files

Two flat text files supply the catalogs that the program loads at startup:

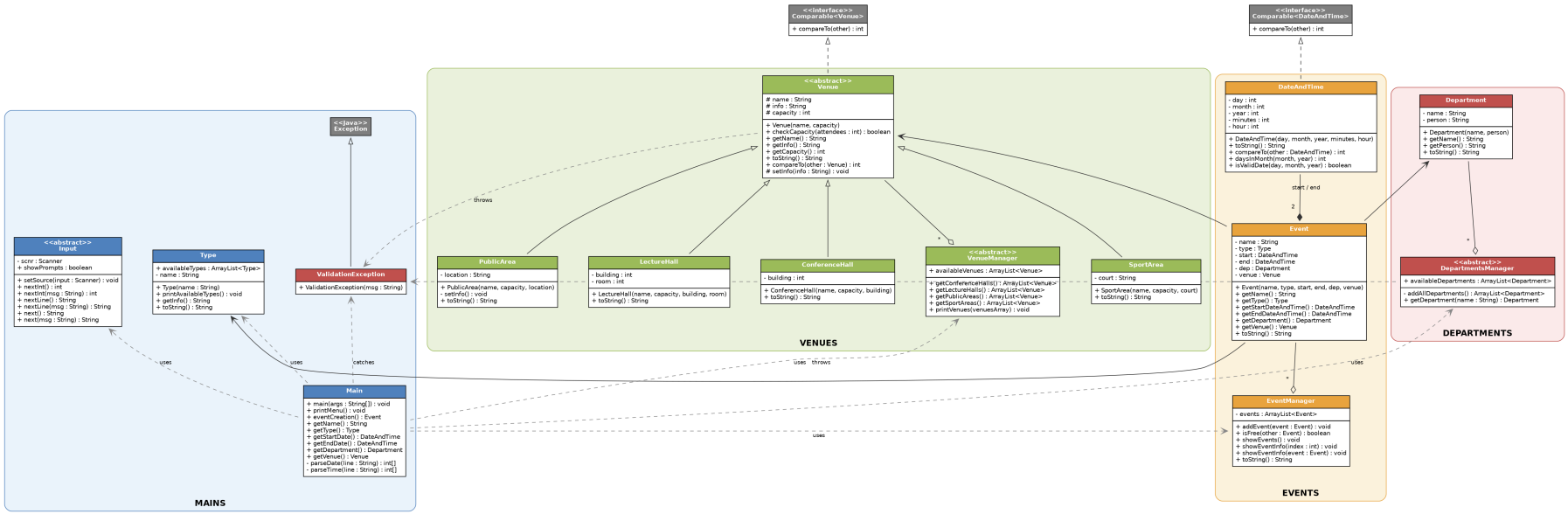
File	Format & Purpose
Departments.txt	One department per line, comma-separated: NAME, PERSON_IN_CHARGE. Loaded into DepartmentsManager.availableDepartments.
Venues.txt	First two lines are a header comment and are skipped. Each remaining line begins with a type code: 1=ConferenceHall, 2=LectureHall, 3=SportArea, 4=PublicArea. The remaining fields on the line vary by type. Loaded into VenueManager.availableVenues.
Test.txt	Optional input script used in file mode. Each line is one input the user would have typed.

3. UML Class Diagram

The diagram on the next page summarizes the static structure of the project. Solid arrows with hollow triangle heads denote inheritance (extends); dashed arrows with hollow triangle heads denote interface realization (implements); diamonds denote aggregation (the manager classes hold a list of items); plain solid arrows denote one-way association (an Event references a Type, a Department, and a Venue); dashed arrows denote dependency (Main uses several other classes).

The PlantUML source for this diagram is shipped alongside this document as EventManagementSystem.puml so it can be re-rendered or pasted into any PlantUML tool (e.g. plantuml.com, VS Code's PlantUML extension)

Figure 1 — Class Diagram



Bader's Event Management System — full class diagram (4 packages, 14 classes / interfaces).

4. Class-by-Class Reference

4.1 Package MAINS

Main

The entry point. main() decides whether to run from a file or from the keyboard, prints the menu when in manual mode, and dispatches the C / SE / SI / Q commands. The static helpers (getName, getType, getStartDate, getEndDate, getDepartment, getVenue) collect each piece of an event in turn, validating along the way. The private parseDate and parseTime helpers split DD MM YYYY and HH MM strings into integer arrays.

Input (abstract utility class)

A thin, defensive wrapper around Scanner. It exposes nextInt, nextLine, and next, each in two overloaded forms (with and without a prompt). Three design choices are worth highlighting:

- Every method loops until valid input is received — InputMismatchException and other Exceptions are caught and the user is asked to try again.
- setSource swaps the underlying Scanner at runtime, which is what enables the file-driven mode in Main.
- The showPrompts flag suppresses on-screen prompts when reading from a script file, keeping the file-mode output clean.

Type

Represents an event classification (Academic / Religious / Sport / Social). The four allowed values are constructed once into a static ArrayList<Type> via createTypesArray, so all of the rest of the code can refer to a closed, well-known set. printAvailableTypes renders the choices as a numbered menu, and getInfo returns a grammatically-correct description ("This is an Academic Event" vs. "This is a Sport Event").

ValidationException

A custom checked exception extending java.lang.Exception. It is thrown whenever business rules are violated (unknown department, invalid menu selection, capacity exceeded) and is caught at the user-interaction layer in Main, where it is converted into a friendly retry message.

4.2 Package DEPARTMENTS

Department

A simple value object: a department has a name and a person in charge. The toString override returns a sentence such as "Malak Baslyman is responsible for ICS Department".

DepartmentsManager (abstract utility class)

Loads Departments.txt at class-loading time into the static availableDepartments list. The split pattern \\s*,\\s* tolerates spaces around the comma. getDepartment(name) does a linear scan and either returns a Department or throws ValidationException with a clear message.

4.3 Package VENUES

Venue (abstract base class, implements Comparable<Venue>)

Defines what every venue has in common: a name, a capacity, and a free-text description (info). The checkCapacity(attendees) method is the central business rule — it returns true if the venue can fit the attendees, otherwise it throws ValidationException. compareTo compares by name, which lets venues be sorted alphabetically without each subclass repeating the logic.

ConferenceHall, LectureHall, SportArea, PublicArea

Each subclass adds the fields specific to its kind (e.g. building number for a Conference Hall; building and room for a Lecture Hall; court type for a Sport Area; location string for a Public Area). Each constructor calls super(name, capacity) and then sets a tailored info string via super.setInfo(...). Each one overrides toString to render itself in a list, which keeps polymorphic printing easy.

VenueManager (abstract utility class)

Mirror of DepartmentsManager but for venues. addAllVenues parses Venues.txt using a Scanner with a custom delimiter that treats both commas and newlines as separators. The leading integer on each line is a tag (1–4) that selects which subclass to instantiate. Four filter methods (getConferenceHalls, getLectureHalls, getPublicAreas, getSportAreas) return only venues of one runtime type via instanceof.

4.4 Package EVENTS

DateAndTime (implements Comparable<DateAndTime>)

Stores day, month, year, hour, and minutes. The constructor validates aggressively: it calls isValidDate (which uses daysInMonth and a leap-year rule) and rejects out-of-range hours/minutes. compareTo walks the fields from year down to minute, returning the first non-zero difference, which is exactly the natural chronological order.

Event

An immutable bundle of an event's six attributes: name, type, start, end, department, and venue. The constructor enforces start < end via DateAndTime.compareTo and throws IllegalArgumentException otherwise.

EventManager

The runtime registry of created events. Two algorithms live here:

- addEvent inserts the new event in chronological order using a single linear scan — it walks until it finds the first existing event whose start is later than the new event's start, then inserts before it (otherwise appends). This keeps the list sorted by start time without ever calling Collections.sort.
- isFree(other) is the venue-clash check. A clash exists if and only if (a) the two events overlap in time and (b) they use the same venue. The overlap rule is the classic interval-overlap formula: end_existing > start_new AND start_existing < end_new.

The two showEventInfo overloads use Java method overloading to accept either a list index or an Event reference; the index-based version delegates to the reference-based version after bounds-checking.

5. OOP Concepts Demonstrated

The project deliberately covers the major OOP topics from ICS 108. Each concept below maps to a concrete location in the code.

Concept	Where it appears
Encapsulation	All fields are private (or protected for inheritance). Access is through getters such as Event.getName() or Venue.getCapacity().
Inheritance	ConferenceHall, LectureHall, SportArea, and PublicArea each extend Venue and reuse its name / capacity / info handling.
Abstract classes	Venue is abstract because it has no meaningful default toString and is never instantiated directly. DepartmentsManager and VenueManager are abstract utility classes (no instances are ever needed).
Polymorphism	EventManager.showEvents iterates Event objects whose venue field is a Venue reference. Each subclass's toString is dispatched at runtime.
Interfaces	Venue implements Comparable<Venue> and DateAndTime implements Comparable<DateAndTime>, allowing them to be ordered by the standard Java sort/compare contract.
Method overloading	Input.nextInt() and Input.nextInt(String). EventManager.showEventInfo(int) and showEventInfo(Event).
Method overriding	Every Venue subclass overrides toString. Event and Department also override toString. ValidationException implicitly uses Exception's machinery via super(msg).
Custom exceptions	ValidationException extends Exception; thrown by DepartmentsManager.getDepartment, Venue.checkCapacity, and the menu loop in Main.
Static collections	Type.availableTypes, DepartmentsManager.availableDepartments, and VenueManager.availableVenues are loaded once at class load and reused.
File I/O	Both manager classes read text files via Scanner + FileInputStream inside try-with-resources, so handles close even on exceptions.
Generics (consumption)	ArrayList<Event>, ArrayList<Venue>, Comparable<Venue>, Comparable<DateAndTime> — generics are consumed throughout.

6. Sample Workflow

The Test.txt script that ships with the project demonstrates the typical path through the program. It creates four valid events of different types and then asks to view them. A condensed trace:

```
C                       <- start a new event
Aramco Football         <- name
3                       <- Sport (from the type menu)
1 1 2000                <- start date  (1 Jan 2000)
14 00                   <- start time  (14:00)
2 1 2000                <- end date    (2 Jan 2000)
16 00                   <- end time    (16:00)
AE                      <- Aerospace Engineering Dept.
1                       <- venue category: Sport Area
1                       <- 1st sport area: Main Stadium
400                     <- expected attendees
...                     <- three more events follow
SE                      <- show all events
SI                      <- show event by index
1
Q                       <- quit
```

After the four events are added, SE prints a sorted table of names, start dates, and end dates. SI prints a single-event detail block including venue capacity and the responsible person.

6.1 Clash Detection in Action

If the user tries to add a second event in the Main Stadium that overlaps the first one in time, EventManager.addEvent will detect the clash via isFree() and print a message such as:

```
Couldn't Add Event "Volleyball Tournament" Due To Date And Time, and Venue Clash
```

The event is silently dropped instead of corrupting the schedule, and control returns to the menu.

7. Error Handling Strategy

The program is layered so each kind of error is caught at the most useful place:

Layer	What it does on a failure
Input.next* methods	Catch InputMismatchException and re-prompt. The user is never bounced out of the menu by a typo.
DateAndTime constructor	Throws IllegalArgumentException for impossible dates (e.g. 31 Feb) or out-of-range times. Caught in Main during event creation.
Event constructor	Throws IllegalArgumentException if start ≥ end. Caught in Main, which then announces "Event was not created" and returns to the menu.
Venue.checkCapacity	Throws ValidationException when attendees exceed capacity. The Main loop catches it and offers another try.
DepartmentsManager.getDepartment	Throws ValidationException for unknown department names; same recovery path.
EventManager.addEvent	Refuses overlapping events in the same venue, prints a message, and continues.
File loading	Departments.txt and Venues.txt are read inside try blocks; a FileNotFoundException prints a clear message instead of crashing.

8. How to Build and Run

From the project root (where Departments.txt and Venues.txt live), compile every package and then run Main:

```
javac MAINS/*.java DEPARTMENTS/*.java EVENTS/*.java VENUES/*.java
java MAINS.Main
```

To run with the test script instead of the keyboard, edit the fileName variable in Main.main to the script's path (e.g. "Test.txt") and re-run. If the file is missing, the program automatically falls back to manual entry and re-enables prompts, so the same binary works in both modes.

8.1 File Layout

```
project/
├── MAINS/        Main.java, Input.java, Type.java, ValidationException.java
├── DEPARTMENTS/  Department.java, DepartmentsManager.java
├── EVENTS/       DateAndTime.java, Event.java, EventManager.java
├── VENUES/       Venue.java, ConferenceHall.java, LectureHall.java,
│                 PublicArea.java, SportArea.java, VenueManager.java
├── Departments.txt
├── Venues.txt
└── Test.txt
```

9. Possible Extensions

- Persist created events back to disk so they survive between runs.
- Add a delete-event command and a search-by-name command.
- Replace the linear-scan insertion in EventManager.addEvent with a TreeSet keyed on start time for O(log n) inserts.
- Allow each department to load its own list of preferred venues from a config file.
- Add a unit-test suite (e.g. JUnit) targeting DateAndTime validation, EventManager.isFree, and Venue.checkCapacity.

— End of Document —